

Every query below was executed against MySQL 8.4, To reproduce:

```
npm run db:up

docker exec -i -e MYSQL_PWD=rootpassword eventplanner-mysql \
mysql -uroot < BONUS-SECTION/01-schema-and-data.sql

docker exec -i -e MYSQL_PWD=rootpassword eventplanner-mysql \
mysql -uroot --table < BONUS-SECTION/02-answers.sql
```

File	Contents
BONUS-SECTION/01-schema-and-data.sql	Tables and the sample data exactly as given
BONUS-SECTION/02-answers.sql	Q1, Q2 and Q4
BONUS-SECTION/ BONUS-SECTION.pdf	This document
README.md	The same content in MD form

A note on the numbering: the assessment paper goes Q1 → Q2 → Q4. There is no Q3 in the document. All three questions that were asked are answered here.

Before writing anything, I noticed Four properties of the supplied sample data that determine whether a query is correct or merely plausible:

1. Two different designations on the same date:

EMP008 (Henry Walsh) has both Associate Developer and Mid Developer effective on the same date (2024-06-01):

txn_id	designation	effective_date
T022	Associate Developer	2024-06-01
T023	Mid Developer	2024-06-01

ORDER BY effective_date DESC alone does not identify a single row here. The answer becomes whichever row the optimizer reaches first, which can change with the MySQL version or the chosen plan. Every query below adds txn_id as a tiebreak, on the reasoning that a higher transaction id was recorded later and therefore represents the later decision. It is a stated assumption rather than a fact the schema guarantees; if the business rule were different, this is the one line that would change.

2. Rows are not stored in chronological order:

EMP005 (Eva Chen):

txn_id	designation	effective_date
T012	Senior Developer	2024-06-15
T013	Mid Developer	2024-03-01
T014	Senior Developer	2024-11-20

Anything that reaches for “the newest row by `txn_id`”, or worse, relies on insertion order, gets Eva wrong. Ordering must be driven by `effective_date`.

3. Duplicate rows:

EMP003 and EMP006 each have two rows with the same designation *and* the same date. Harmless for the answer, but they mean a `COUNT(*)`-based approach to “how many promotions” would over-count.

4. Resigned is a designation value, not an absence of one.

EMP007 (Grace Kim) resigned in 2023 and was rehired in 2024. A naive filter on “developer-ish” designations would silently drop her history

Q1. Current designation for every employee:

Requirement: Return the current designation of every employee, defined as the designation from their most recent `effective_date``.

Output columns: `emp_id` | `emp_name` | `current_designation`

Approach: Rank each employee’s rows newest-first and keep the top one. A window function does this in a single pass over the table, without the self-join that the classic formulation needs.

Query:

```
SELECT emp_id, emp_name, designation AS current_designation
FROM (
  SELECT
    emp_id,
    emp_name,
    designation,
    ROW_NUMBER() OVER (
      PARTITION BY emp_id
      ORDER BY effective_date DESC, txn_id DESC -- tiebreak: see above
    ) AS rn
  FROM emp_designation_log
) AS ranked
WHERE rn = 1
ORDER BY emp_id;
```

OUTPUT:

emp_id	emp_name	current_designation
EMP001	Alice Johnson	Senior Developer
EMP002	Bob Martinez	Mid Developer
EMP003	Carol Smith	Mid Developer
EMP004	David Lee	Mid Developer
EMP005	Eva Chen	Senior Developer
EMP006	Frank Patel	Mid Developer
EMP007	Grace Kim	Mid Developer
EMP008	Henry Walsh	Mid Developer
EMP009	Irene Novak	Senior Developer

Interpretation of results: EMP002 (Bob Martinez) is Mid Developer, not Senior Developer. He was demoted on 2024-09-20, and “current” means most recent, not highest. EMP005 (Eva Chen) is Senior Developer via her 2024-11-20 row, which is the case that punishes any ordering by txn_id. EMP007 (Grace Kim) is Mid Developer, not Resigned, The rehire being the latest state.

Alternative formulation without window functions

For environments predating MySQL 8.0, the same result is obtained by joining each employee to their own maximum date. The tiebreak must then be applied twice. Verified to return identical results to the query above.

```
SELECT d.emp_id, d.emp_name, d.designation AS current_designation
FROM emp_designation_log AS d
JOIN (
  SELECT emp_id, MAX(effective_date) AS max_date
  FROM emp_designation_log
  GROUP BY emp_id
) AS latest
ON latest.emp_id = d.emp_id
AND latest.max_date = d.effective_date
WHERE d.txn_id = (
  SELECT MAX(d2.txn_id)
  FROM emp_designation_log AS d2
  WHERE d2.emp_id = d.emp_id AND d2.effective_date = d.effective_date
)
ORDER BY d.emp_id;
```

Q2. Designation timeline (previous / current / next)

Requirement: For every row in the designation table, return the designation held immediately before and immediately after that row for the same employee, with 'NULL' where no such row exists.

Output columns:

emp_id | effective_date | previous_designation | designation | next_designation

Approach: LAG and LEAD over the same partition. Both must use an *identical* ordering clause. If they disagreed, a row's next_designation would not match the following row's designation. Naming the window once with a WINDOW clause makes that structurally impossible rather than merely intended.

Query:

```
SELECT
  emp_id,
  effective_date,
  LAG(designation) OVER w AS previous_designation,
  designation,
  LEAD(designation) OVER w AS next_designation
FROM emp_designation_log
WINDOW w AS (PARTITION BY emp_id ORDER BY effective_date, txn_id)
ORDER BY emp_id, effective_date, txn_id;
```

LAG/LEAD return NULL at the partition edges with no special handling, which is exactly what the question asks for.

Output:

emp_id	effective_date	previous_designation	designation	next_designation
EMP001	2024-02-01	NULL	Associate Developer	Mid Developer
EMP001	2024-02-05	Associate Developer	Mid Developer	Senior Developer
EMP001	2024-02-10	Mid Developer	Senior Developer	NULL
EMP002	2024-05-02	NULL	Mid Developer	Senior Developer
EMP002	2024-07-15	Mid Developer	Senior Developer	Mid Developer
EMP002	2024-09-20	Senior Developer	Mid Developer	NULL
EMP003	2024-08-06	NULL	Mid Developer	Mid Developer
EMP003	2024-08-06	Mid Developer	Mid Developer	NULL
EMP004	2024-01-10	NULL	Associate Developer	Associate Developer
EMP004	2024-04-10	Associate Developer	Associate Developer	Mid Developer
EMP004	2024-09-10	Associate Developer	Mid Developer	NULL
EMP005	2024-03-01	NULL	Mid Developer	Senior Developer
EMP005	2024-06-15	Mid Developer	Senior Developer	Senior Developer
EMP005	2024-11-20	Senior Developer	Senior Developer	NULL
EMP006	2024-01-01	NULL	Associate Developer	Mid Developer
EMP006	2024-05-10	Associate Developer	Mid Developer	Mid Developer
EMP006	2024-05-10	Mid Developer	Mid Developer	NULL
EMP007	2023-03-03	NULL	Senior Developer	Resigned
EMP007	2023-06-30	Senior Developer	Resigned	Associate Developer
EMP007	2024-01-15	Resigned	Associate Developer	Mid Developer

emp_id	effective_date	previous_designation	designation	next_designation
EMP007	2024-07-15	Associate Developer	Mid Developer	NULL
EMP008	2024-06-01	NULL	Associate Developer	Mid Developer
EMP008	2024-06-01	Associate Developer	Mid Developer	NULL
EMP009	2024-09-01	NULL	Senior Developer	NULL

Interpretation of results: Two things the output makes visible. EMP005 is now in date order (2024-03-01 first) despite its storage order. The window ordering fixed it. And EMP003 and EMP006 produce steps in which the previous and current designations are equal (Mid → Mid), which is a correct rendering of duplicate rows.

Variant: transitions only

If the requirement is a promotion history rather than a transaction history, the no-change rows can be dropped. This requires two window passes, not one pass plus a filter. Filtering the completed result set removes rows without recomputing `LAG` and `LEAD` on the rows that survive: dropping EMP004's 2024-04-10 row leaves the preceding row asserting next_designation = Associate Developer, while the next row actually displayed is Mid Developer. A WHERE clause at the outer level is in any case rejected, window functions being evaluated after WHERE.

MySQL does not implement IS DISTINCT FROM, so the null-safe equality operator `<=>` expresses both the comparison and the retention of each employee's first row in one predicate.

```
WITH marked AS (
  SELECT
    txn_id, emp_id, designation, effective_date,
    LAG(designation) OVER (
      PARTITION BY emp_id ORDER BY effective_date, txn_id
    ) AS prev_raw
  FROM emp_designation_log
),
changes_only AS (
  SELECT txn_id, emp_id, designation, effective_date
  FROM marked
  WHERE NOT (prev_raw <=> designation)  -- null-safe: keeps each employee's first row
)
SELECT
  emp_id,
  effective_date,
  LAG(designation) OVER w AS previous_designation,
  designation,
  LEAD(designation) OVER w AS next_designation
FROM changes_only
WINDOW w AS (PARTITION BY emp_id ORDER BY effective_date, txn_id)
ORDER BY emp_id, effective_date, txn_id;
```

this removes T008, T010, T014 and T017, returning 20 rows. Note that T010 is removed although its date differs from the preceding row: a repeated designation is a non-transition whether or not the date repeats. Whether to apply it is a product decision, not a SQL one. The query above answers the question as asked (*every row*); this variant answers what a timeline screen probably wants.

Q4. Designation held at the time of each allocation

Requirement: For each project allocation, return the designation the employee held on the allocation start date.

Output columns:

allocation_id | emp_id | emp_name | project_name | allocated_role | allocation_start |
designation_at_allocation

The reasoning, before the SQL

The designation table records only the date on which a designation began. There is no end date, so so each designation's validity interval is implicit: it runs until the next row for that employee, and the last row runs forever.

That gives two ways to answer:

1. **Look backwards from the allocation date.** For each allocation, find the employee's latest designation row with `effective_date <= allocation_start`. One row in, at most one row out.
2. **Build the intervals first.** Use `LEAD` to derive each designation's end date, then range-join the allocation into the interval that contains it.

Both are correct. I chose (1) as the primary answer because it states the requirement directly, "the designation in force on this date" and because it is index-friendly: `(emp_id, effective_date)` turns each lookup into a backwards range scan stopping at the first row. Approach (2) is better when you need *all* the intervals for something else anyway; it is given below as an alternative.

The three stated considerations:

1. **An employee may have held multiple designations: Only one was active on allocation_start:** The subquery orders by `effective_date DESC, txn_id DESC` and applies `LIMIT 1`, selecting exactly the latest designation in force on that date. The `txn_id` term resolves the same-date ambiguity described in section 1.
2. **The designation table stores no end date:** No end date is derived. The predicate `effective_date <= allocation_start` combined with descending order and `LIMIT 1` locates the applicable row without materializing intervals. The comparison is inclusive: a designation effective 2024-08-06 is in force for an allocation beginning 2024-08-06. This case occurs in the sample data as A005.
3. **An employee may have no designation record before allocation_start:** The allocation is retained and `designation_at_allocation` is `NULL`.

Query:

```
WITH employees AS (
  SELECT emp_id, MAX(emp_name) AS emp_name
  FROM emp_designation_log
  GROUP BY emp_id
)
SELECT
  a.allocation_id,
  a.emp_id,
  e.emp_name,
  a.project_name,
  a.allocated_role,
  a.allocation_start,
  (
    SELECT d.designation
    FROM emp_designation_log AS d
    WHERE d.emp_id = a.emp_id
      AND d.effective_date <= a.allocation_start
    ORDER BY d.effective_date DESC, d.txn_id DESC
    LIMIT 1
  ) AS designation_at_allocation
FROM emp_allocation_log AS a
LEFT JOIN employees AS e ON e.emp_id = a.emp_id
ORDER BY a.allocation_id;
```

Output:

allocation_id	emp_id	emp_name	project_name	allocated_role	allocation_start	designation_at_allocation
A001	EMP001	Alice Johnson	Project Alpha	Developer	2024-02-03	Associate Developer
A002	EMP001	Alice Johnson	Project Beta	Tech Lead	2024-05-01	Senior Developer
A003	EMP002	Bob Martinez	Project Alpha	Developer	2024-05-10	Mid Developer
A004	EMP002	Bob Martinez	Project Gamma	Senior Contributor	2024-09-01	Senior Developer
A005	EMP003	Carol Smith	Project Beta	Developer	2024-08-06	Mid Developer
A006	EMP004	David Lee	Project Delta	Developer	2024-02-01	Associate Developer
A007	EMP005	Eva Chen	Project Alpha	Senior Contributor	2024-04-01	Mid Developer
A008	EMP005	Eva Chen	Project Gamma	Tech Lead	2024-08-01	Senior Developer
A009	EMP006	Frank Patel	Project Delta	Developer	2024-03-01	Associate Developer
A010	EMP007	Grace Kim	Project Beta	Developer	2024-02-01	Associate Developer

allocation_id	emp_id	emp_name	project_name	allocated_role	allocation_start	designation_at_allocation
A011	EMP008	Henry Walsh	Project Alpha	Developer	2024-07-01	Mid Developer
A012	EMP009	Irene Novak	Project Gamma	Senior Contributor	2024-10-01	Senior Developer

A001 returns **Associate Developer**, which is precisely the worked example in the question — Alice joined Project Alpha on 2024-02-03 and only became Mid Developer on 2024-02-05. That is the check that the whole thing is right.

Three rows depend on decisions taken above:

Row	Value	Dependency
A008 Eva Chen, 2024-04-01	Mid Developer	Ordering by effective_date rather than txn_id. Ordering by txn_id returns Mid Developer
A011 Henry Walsh, 2024-07-01	Mid Developer	Non-deterministic without the txn_id tiebreak: two designations share 2024-06-01
A004 Bob Martinez, 2024-09-01	Senior Developer	Mid Developer, if you take his <i>current</i> designation instead of the one in force at the time. His demotion lands on 2024-09-20, nineteen days later

“What happens if an employee has no designation before the allocation_start?”

The condition does not occur in the sample data. No allocation emp_id is absent from the designation log, and no allocation begins before its employee's first designation. The branch is therefore unexercised by the supplied data and so to verify it add two rows that force it. One employee absent from the designation log entirely, and one allocation predating the employee's first

allocation_id	emp_id	allocation_start	designation_at_allocation
A998	EMP001	2024-01-01	NULL
A999	EMP010	2024-05-01	NULL

Both allocations survive. A998 resolves its employee name (**Alice Johnson**), the employee being present in the designation log, she simply had no designation on 2024-01-01. A999 does not, the employee appears nowhere in the designation log at all, so the name is genuinely unrecoverable from these two tables and stays **NULL**. This distinction is the purpose of resolving the name through a separate join.

Alternative formulation: validity intervals

Applicable where designation intervals are required for other reporting.

```
WITH employees AS (
  SELECT emp_id, MAX(emp_name) AS emp_name
  FROM emp_designation_log
  GROUP BY emp_id
),
designation_spans AS (
  SELECT
    emp_id,
    designation,
    effective_date AS valid_from,
    LEAD(effective_date) OVER (
      PARTITION BY emp_id ORDER BY effective_date, txn_id
    ) AS valid_until      -- NULL on the final row = open-ended
  FROM emp_designation_log
)
SELECT
  a.allocation_id, a.emp_id, e.emp_name, a.project_name,
  a.allocated_role, a.allocation_start,
  s.designation AS designation_at_allocation
FROM emp_allocation_log AS a
LEFT JOIN employees AS e
  ON e.emp_id = a.emp_id
LEFT JOIN designation_spans AS s
  ON s.emp_id = a.emp_id
  AND a.allocation_start >= s.valid_from
  AND (s.valid_until IS NULL OR a.allocation_start < s.valid_until)
ORDER BY a.allocation_id;
```

The interval comparison must be half-open. Same-date rows produce spans of zero width, in which `valid_from = valid_until`. The condition `>= from AND < until` excludes them correctly. Substituting `<=` causes an allocation whose start date falls exactly on a repeated date to match two spans and duplicate. Under a closed comparison, A005 returns two rows. The scalar-subquery formulation has no equivalent boundary condition, which is why it is preferred as the primary answer.